

SCHOOL OF ADVANCED TECHNOLOGY

SAT301 FINAL YEAR PROJECT

Machine Learning-Driven Compact Modeling of Advanced Devices

Final Thesis

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Engineering

Student Name: Linfeng Li
Student ID: 2145589
Supervisor: Kain Lu Low
Assessor: Kain Lu Low & Ping Zhang

Abstract

The increasing complexity of semiconductor devices at nanoscale has placed significant demands on the accuracy and scalability of compact models used in electronic design automation. Traditional compact modeling approaches often fall short in capturing the intricate, non-linear characteristics of advanced devices. This thesis proposes a two-stage machine learning-driven framework to generate compact models using TCAD-derived datasets. In the initial stage, a baseline MLP model is implemented on a simplified dataset to verify feasibility. The second stage extends the methodology using more complex datasets and advanced models, including FatGCN, Random Forest, and XGBoost, with performance metrics assessed using MAE and MSE.

Through comparative experimentation, the ensemble models demonstrated superior accuracy, with XGBoost achieving the lowest MAE of 0.0068, while FatGCN showed potential for further improvement through tuning. Evaluation using Id–Vg plots confirms the viability of the ML-derived models, particularly when trained on realistic simulation data. Furthermore, the work highlights the critical role of compact models within DTCO workflows, where precise Id–Vg predictions contribute directly to PPAC (Power, Performance, Area, and Cost) optimization.

Although current results show promising accuracy, the research also identifies the need for model-specific thresholds to assess industrial applicability and recommends high-performance training for further precision gains. Overall, this thesis contributes a modular and scalable methodology for compact modeling that supports integration into circuit simulation and semiconductor design pipelines.

Keywords: Compact modeling; Machine learning; TCAD simulation; ANN; XGBoost; Random Forest

Acknowledgements

I would like to express my sincere gratitude to Professor Kain Lu Low for his invaluable guidance throughout this project. His expertise in semiconductor technologies and their intersection with machine learning—which is also the focus of my major—has been instrumental in shaping the direction of this research. I am particularly grateful for his practical advice on time management and academic writing, drawn from his extensive experience at the doctoral and master's levels.

I would also like to thank Professor Ping Zhang and Professor S. Lam for their insightful suggestions regarding dataset selection, which addressed key issues not explicitly stated in the project scope but critical to its success.

My appreciation also extends to my fellow research group members, Yuhang and Xiang, whose intellectual contributions and collaborative spirit provided a valuable support system during the course of this work.

Contents

Abstract i

Acknowledgements ii

List of Tables v

List of Figures vi

List of Acronyms vii

1 Introduction 1

1.1 Motivation 1

1.2 Aims and Objectives 4

1.3 Literature Review 6

1.3.1 Compact Model in DTCO 6

1.3.2 Traditional Compact modeling 7

1.3.3 Machine Learning Driven Compact Modeling 10

2 Methodology and Results 14

2.1 Methodology 14

2.1.1 ANN (MLE) Model and Basic Demonstration Dataset 15

2.1.2 ANN, Bagging/Boosting Models and Real-World Dataset 20

2.2 Results 28

2.3 Evaluation 31

3 Conclusion and Future Work 32

3.1 Conclusion 32

3.2 Future Work 33
3.2.1 FatGCN Further Tuning 33
3.2.2 Assessment for Current Training Loss and Criteria 34

References **37**

A Machine Learning Implementation **38**

A.1 Multilayer Perceptron (MLE) 38

A.2 Graph Convolutional Network (FatGCN) 38

A.3 Random Forest 39

A.4 XGBoost 40

List of Tables

2.1 Overview of extracted TCAD simulation datasets 16
2.2 Comparison of FatGCN and MLE architectures based on implemented models. 26
2.3 Model Structure of Random Forest and XGBoost models. 28

List of Figures

1.1 Global semiconductor market trends year-over-year. Source: [1] 2
1.2 Moore's Law and continued transistor scaling. 2
1.3 Framework of ML-driven compact modeling 5
1.4 Explanation of DTCO [2] 7
1.5 Evolution chart of the mainstream compact models: ϕ_s-based, V_{th}-based, and Q-based [2] 8
1.6 The procedure of ML compact model development [2] 10
1.7 Pi-NN Model Structure Clarification [3] 12

2.1 Model tuning and optimization procedure for multi-model compact modeling 15
2.2 nFET I_{D}–V_{G} dataset used in the first-stage modeling 17
2.3 General Neural Network Architecture and Symbol Definitions 18
2.4 MLE Model Training procedure & Loss calculation 20
2.5 TCAD Simulation Dataset Preparation 22
2.6 Feature Importance Analysis 23
2.7 MSE descent curve for the basic MLE framework. 29
2.8 Comparison of MAE convergence among different models in the second framework. 30
2.9 Comparison of $I_{\text{D}} - V_{\text{G}}$ prediction accuracy between FatGCN and MLE models. 31

3.1 Referenced FatGCN model tuning procedure [4] 34

List of Acronyms

Term	Initial components of the term
FET	Field Effect Transistor
GAAFET	Gate-All-Around Field Effect Transistor
FinFET	Fin Field Effect Transistor
TCAD	Technology Computer-Aided Design
SPICE	Simulation Program with Integrated Circuit Emphasis
ML	Machine Learning
ANN	Artificial Neural Network
MLP	Multi-Layer Perceptron
GCN	Graph Convolutional Network
FatGCN	Fat Graph Convolutional Network
MSE	Mean Squared Error
MAE	Mean Absolute Error
PINN	Physics-Informed Neural Network
RF	Random Forest
SVM	Support Vector Machine
DTCO	Design-Technology Co-Optimization
HCD	Hot Carrier Degradation
BTI	Bias Temperature Instability
VGS	Gate-Source Voltage
VDS	Drain-Source Voltage
Id	Drain Current
Vg	Gate Voltage
HPC	High-Performance Computing

Chapter 1

Introduction

1.1 Motivation

The semiconductor industry is undergoing a profound transformation, driven by accelerating technological advancements and surging global demand across various sectors. According to the 2024 Semiconductor Industry Outlook [1], the global semiconductor market reached a revenue of US $627 billion in 2023, marking a 19% year-over-year growth and surpassing previous projections. This strong momentum is projected to continue, with expected revenues of US$697 *billion in* 2025 *and a milestone target of US*$1 trillion by 2030. If this compound annual growth rate (CAGR) of approximately 7.5% sustains, the market is on track to double again by 2040, potentially reaching US$2 trillion. The implications of such exponential growth extend beyond mere market figures, signaling a fundamental shift in how semiconductor technologies are positioned at the core of modern innovation.

Market performance, however, has not been uniform across segments. In particular, companies engaged in the development of AI-optimized integrated circuits have significantly outperformed those in more traditional areas such as automotive, consumer electronics, and communication chips. This discrepancy illustrates the growing importance of generative AI, edge computing, and data-centric architectures, which place stringent demands on device performance, energy efficiency, and integration density [1, 5]. Consequently, the industry's ongoing adherence to Moore's Law—despite its increasing complexity—is

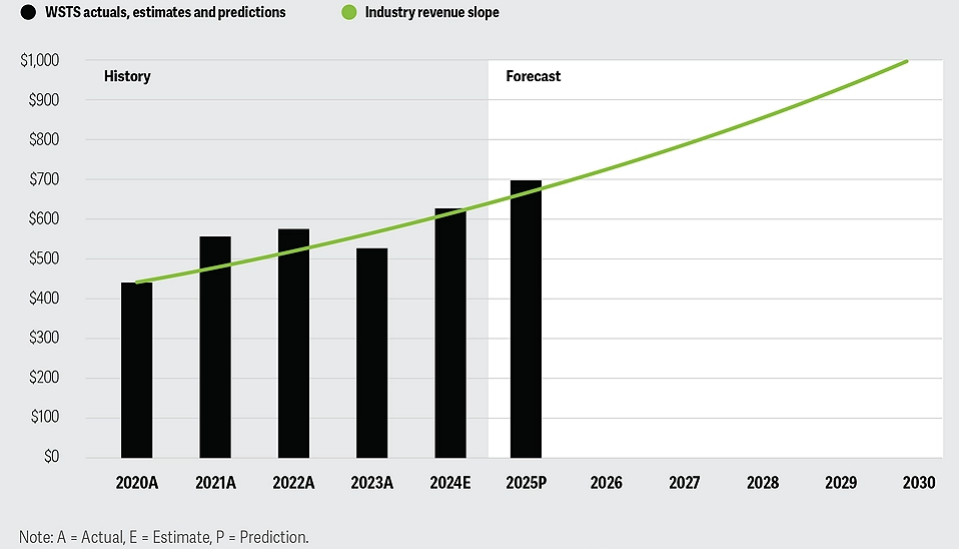


Figure 1.1: Global semiconductor market trends year-over-year. Source: [1]

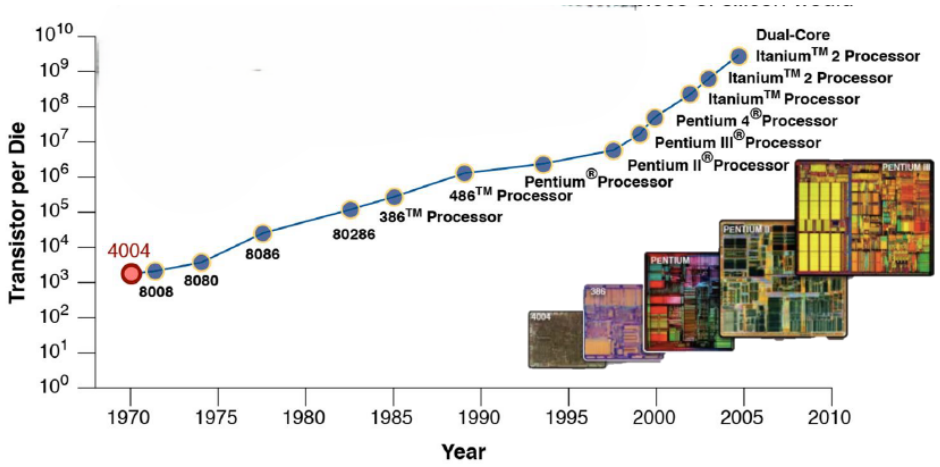


Figure 1.2: Moore's Law and continued transistor scaling.

being tested by the physical and design limitations associated with scaling below the 5 nm technology node (Figure 1.2).

As device dimensions shrink and architectures become more intricate, maintaining precise control over electrostatic integrity, thermal management, and variability becomes increasingly challenging. Advanced transistor structures such as FinFETs, Gate-All-Around FETs (GAAFETs), and 2D-material-based devices require accurate characterization of short-channel effects, quantum confinement, gate leakage, and other nanoscale phenomena [6, 7]. In the face of these challenges, compact device models play an indispensable role. They enable efficient simulation of complex integrated circuits by abstracting the behavior of physical devices into simplified mathematical formulations. Within the frame-

work of design-technology co-optimization (DTCO), compact models serve as the critical link between device fabrication and electronic design automation (EDA), facilitating accurate performance prediction, power optimization, and area-efficiency analysis.

Traditionally, compact modeling approaches have followed two paradigms: empirical models and physics-based models. Empirical models are typically derived through curve fitting of experimental data, making them computationally efficient and straightforward to implement. However, they often lack generalizability and can exhibit non-physical behavior outside of trained regimes. On the other hand, physics-based models—built upon first principles such as Poisson's equation and drift-diffusion transport—offer strong physical fidelity and predictive capability. Yet, as devices scale further, these models demand increasingly complex formulations, especially when quantum mechanical effects, parasitic interactions, and reliability concerns like bias temperature instability (BTI) and hot carrier degradation (HCD) must be incorporated. As a result, the development cycle becomes time-consuming, and the simulation cost rises substantially, limiting their practicality in large-scale design flows [2].

Recent advancements in machine learning (ML) have opened promising avenues for compact modeling by offering a data-driven alternative capable of balancing accuracy, efficiency, and adaptability. In particular, artificial neural networks (ANNs) have attracted considerable interest due to their universal approximation capabilities, enabling them to model intricate, non-linear relationships inherent in semiconductor devices [8, 9]. Unlike traditional look-up table (LUT) methods, which often require prohibitively large datasets and suffer from interpolation artifacts, ANNs provide scalable, smooth, and generalizable solutions. Neural network models can achieve high accuracy with reduced data complexity, offering continuous, infinitely differentiable functions that are highly desirable for ensuring numerical stability in circuit simulation environments.

Furthermore, while table-based models are inherently limited in their ability to extrapolate beyond the training dataset—often resulting in discontinuous or oscillatory behavior—the neural network framework enables improved generalization performance through careful training and regularization techniques. This robustness against noise and data sparsity is particularly valuable for modeling emerging devices for which large-scale experimental datasets may not be readily available. Moreover, neural networks demonstrate

excellent smoothness in the resulting device characteristics, an essential property for convergence in very-large-scale integration (VLSI) circuit simulations, where the presence of non-smooth model outputs can cause simulation divergence or instability [8].

Despite these advantages, purely data-driven models have historically been criticized for lacking physical interpretability. A key concern is that unconstrained neural networks may produce unphysical results—such as non-zero drain current at zero bias conditions—thereby compromising their reliability in real-world applications. To address this limitation, recent research has focused on embedding physical laws and constraints directly into the machine learning framework. One notable example is the Physics-Informed Neural Network (Pi-NN), which incorporates known physical relationships into the architecture and training process of the network. This hybrid approach effectively marries the expressiveness of ML with the rigor of physics-based modeling, producing compact models that are accurate, physically consistent, and computationally efficient.

This thesis is motivated by the urgent need for next-generation compact modeling techniques that can support the semiconductor industry's push toward higher performance, lower power consumption, and increasing integration complexity. Specifically, it proposes a machine learning-driven compact modeling framework that leverages Technology Computer-Aided Design (TCAD) simulation data in conjunction with advanced ML algorithms—such as neural networks, Support Vector Machines (SVMs), and Decision Trees. The objective is to develop models that not only achieve high predictive accuracy and smoothness but also respect the underlying physics of device operation. By enabling rapid and realistic simulation of novel transistor structures, this approach aims to accelerate innovation in electronic design automation (EDA) and facilitate the industry's transition toward emerging device architectures and advanced technology nodes [2, 8, 9].

1.2 Aims and Objectives

The primary aim of this project is to develop and implement a machine learning-driven compact modeling framework for advanced semiconductor devices, with a particular focus on predictive accuracy, physical consistency, and computational viability. As semicon-

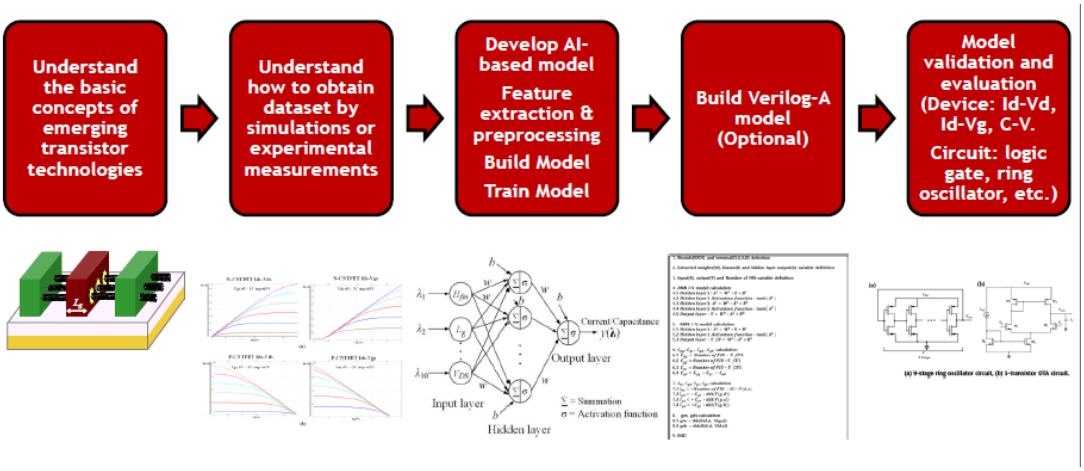


Figure 1.3: Framework of ML-driven compact modeling

ductor devices enter the nanoscale regime, traditional modeling techniques struggle to capture the increasingly complex physical phenomena involved. This project addresses this gap by combining domain-informed neural networks with ensemble learning techniques, trained on both simple synthetic datasets and realistic TCAD simulation data.

To accomplish this aim, the research is structured around the framework shown in Figure 1.3, and is designed to meet the following specific objectives:

1. **Establish Feasibility with Prototype Modeling**

   Construct a baseline compact modeling framework using a simple dataset and a multi-layer perceptron (MLP) to evaluate the feasibility of machine learning for predicting I_D-V_G behavior. This stage serves to validate core concepts such as model convergence, basic tuning, and error analysis using Mean Squared Error (MSE).

2. **Integrate Realistic TCAD Data**

   Employ TCAD-simulated datasets that capture physically accurate device behavior under varying gate and drain voltages. These data provide a more realistic and challenging foundation for training robust compact models reflective of modern device architectures [10, 8].

3. **Apply and Compare Advanced ML Algorithms**

   Implement and benchmark multiple machine learning techniques—including enhanced neural networks (MLP, FatGCN), ensemble models (Random Forest, XG-

Boost), and graph-based models—on the realistic TCAD dataset. Model performance is evaluated using both MSE and Mean Absolute Error (MAE) to ensure precision and robustness.

4. **Enforce Physical Consistency**
   Mitigate unphysical outputs in model predictions (e.g., negative or non-zero current at zero bias) by incorporating device physics constraints, guided by prior work in physics-informed neural networks (PINNs) and hybrid loss function design [11].

5. **Optimize for Training Efficiency and Scalability**
   Improve model training performance through tuning procedures, parameter pruning, and dataset refinement. This ensures the framework remains scalable for more complex device structures and practical in resource-constrained environments, such as on-chip or real-time simulation platforms [8].

6. **Support Cross-Device Generalizability**
   Design the modeling pipeline to be flexible and generalizable across different device types, such as FinFETs, GAAFETs, and 2D-material transistors, enabling broad adoption across future semiconductor technologies [10, 11].

7. **Evaluate Through Visual and Quantitative Metrics**
   Use both quantitative error metrics (MAE/MSE) and visual comparisons of I_D-V_G plots to evaluate prediction fidelity. This dual evaluation approach supports interpretability and highlights modeling strengths or limitations across voltage regimes.

1.3 Literature Review

1.3.1 Compact Model in DTCO

The development of accurate and scalable compact models is a cornerstone in semiconductor device design, particularly within Design-Technology Co-Optimization (DTCO) workflows. Shown in Figure 1.3, in this workflow, compact models function as the bridge between physical device fabrication and electronic circuit design, enabling accurate and

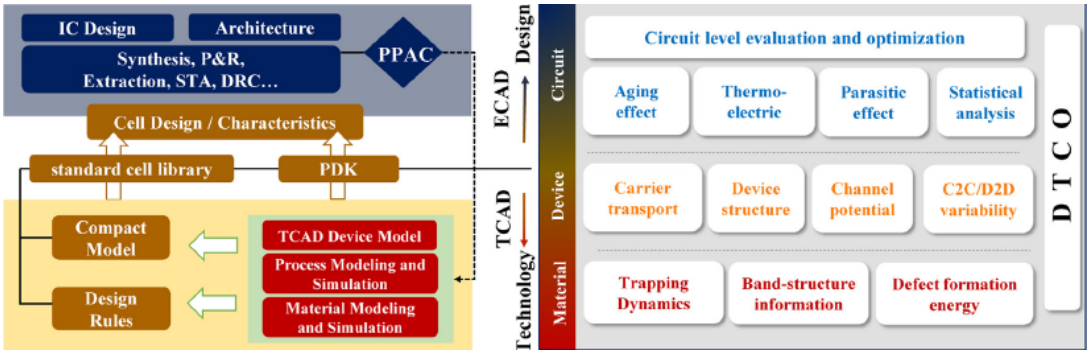


Figure 1.4: Explanation of DTCO [2]

efficient simulations essential for power, performance, area, and cost (PPAC) optimization [2]. Through TCAD-to-SPICE methodologies, device characteristics are extracted from Technology Computer-Aided Design (TCAD) simulations and transformed into simulation-ready compact models. To ensure realism and predictive power, advanced TCAD simulations often integrate complex reliability concerns such as parasitic effects, gate leakage, hot carrier degradation (HCD), bias temperature instability (BTI), and quantum mechanical effects [2].

1.3.2 Traditional Compact modeling

Traditional compact modeling forms the backbone of circuit-level simulation in integrated circuit (IC) design, having undergone a rich evolution since its inception in the mid-20th century as shown in Figure 1.5. The earliest forms of compact models, such as the Ebers-Moll and Gummel-Poon models for bipolar transistors, laid the groundwork for modeling charge transport and current flow in semiconductor devices. With the advancement of field-effect transistors (FETs), compact modeling progressed further with the development of the Pao-Sah model in 1966, which introduced a more rigorous physics-based description of carrier drift and diffusion behavior across all operational regimes [2]. These early models relied heavily on fundamental physical principles, often centered around Poisson's equation and drift-diffusion (DD) theory, and typically required iterative numerical methods to solve for internal quantities such as the ϕ_s and V_{th}. [12] [2]

As semiconductor technology rapidly advanced and entered the submicron and eventually nanoscale regimes, the demand for more accurate and scalable compact models led to the

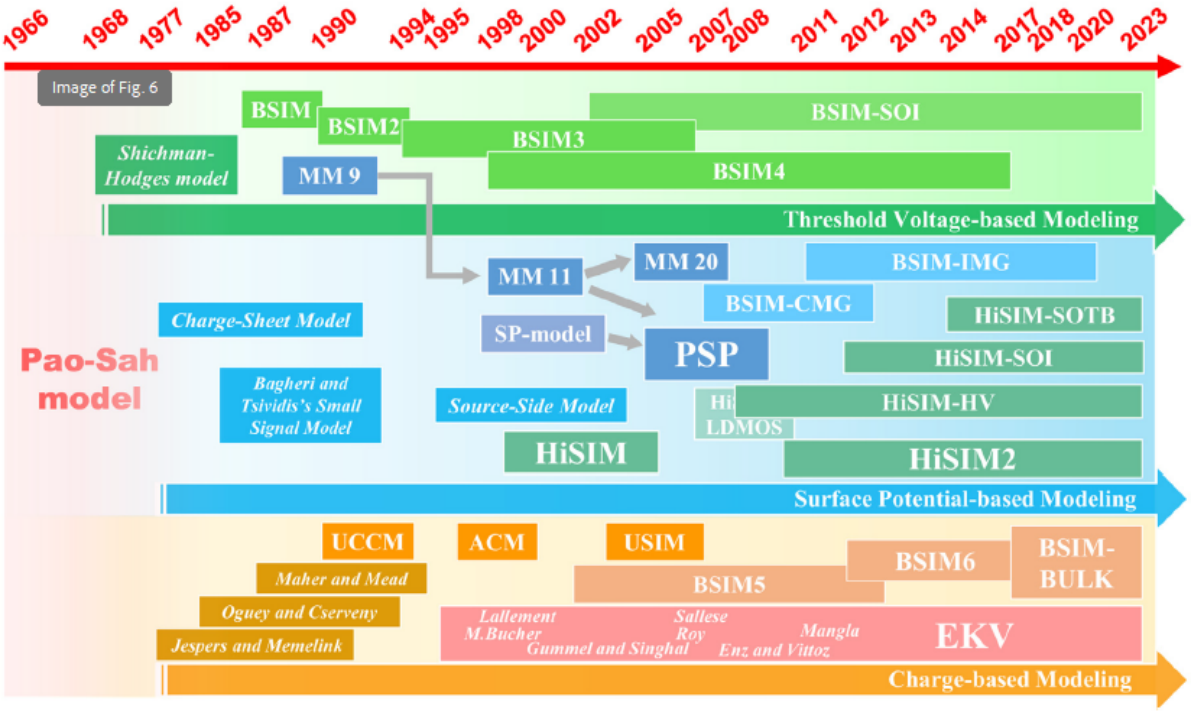


Figure 1.5: Evolution chart of the mainstream compact models: ϕ_s-based, V_{th}-based, and Q-based [2]

creation of increasingly sophisticated frameworks. This included threshold-voltage-based models, surface-potential-based models, and charge-based models, all of which aimed to simplify the physics while retaining computational tractability. Prominent among these are the BSIM (Berkeley Short-channel IGFET Model), PSP (Penn State Philips), and HiSIM (Hiroshima-university STARC IGFET Model) families of models that serve as industrial standards today for modeling CMOS behavior [2][5][7]. These models are carefully parameterized and extensively calibrated against experimental data, offering good trade-offs between accuracy and simulation efficiency. Their implementation into SPICE simulators has enabled the development of accurate and reliable design automation tools over the past few decades.

Nonetheless, traditional compact models are not without their limitations, particularly as device dimensions shrink into the sub-5 nm regime. [12] At these scales, physical phenomena such as short-channel effects, mobility degradation, source-to-drain tunneling, and quantum confinement become dominant and increasingly complex to model. The incorporation of such effects typically requires more elaborate physical approximations and introduces a growing number of fitting parameters, which complicates model devel-

opment and calibration [7]. As a result, maintaining the balance between model accuracy, analytical tractability, and simulation efficiency becomes an increasingly difficult task.

The evolution of Technology Computer-Aided Design (TCAD) tools parallels the growth of compact modeling and has significantly contributed to the development of physics-based models. Starting from one-dimensional simulations in the 1960s, TCAD tools progressed to two- and three-dimensional simulations to accommodate the effects of device miniaturization, such as non-uniform doping profiles and parasitic resistance [2]. Tools such as MINIMOS, SUPREM, and PISCES paved the way for today's comprehensive platforms like Synopsys Sentaurus and Silvaco Atlas. These simulators provide detailed insight into carrier transport, quantum effects, thermal phenomena, and more—facilitating a close coupling between physical device exploration and compact modeling. However, as devices approach atomic dimensions, the traditional DD-based TCAD framework struggles to incorporate quantum mechanical behavior with high fidelity without incurring immense computational overhead.

In the post-Moore era, these challenges have surfaced as critical bottlenecks for the physics-engine-based TCAD-to-SPICE compact modeling workflow.[2] These physics-driven models, often characterized as "white-box" approaches, must satisfy a suite of essential criteria—symmetry, continuity, scalability, and computational efficiency—while utilizing as few parameters as possible. However, achieving all of these simultaneously is increasingly infeasible under the constraints imposed by modern nanotechnology. The reliance on simplified assumptions, analytical tractability, and perturbative methods breaks down in the presence of quantum tunneling, discrete doping, random telegraph noise, and strong variability among devices. Furthermore, the growing complexity of fabrication processes and device geometries makes it increasingly difficult to derive compact models that are both accurate and broadly applicable.

Consequently, although traditional compact modeling frameworks have been instrumental in enabling the success of the semiconductor industry thus far, their limitations in the context of aggressively scaled technologies have sparked the search for new paradigms. The immense computational burden of simulating complex physical interactions using TCAD, combined with the impracticality of manually deriving high-fidelity compact models for every novel device structure, underscores the need for a more automated, flexible, and

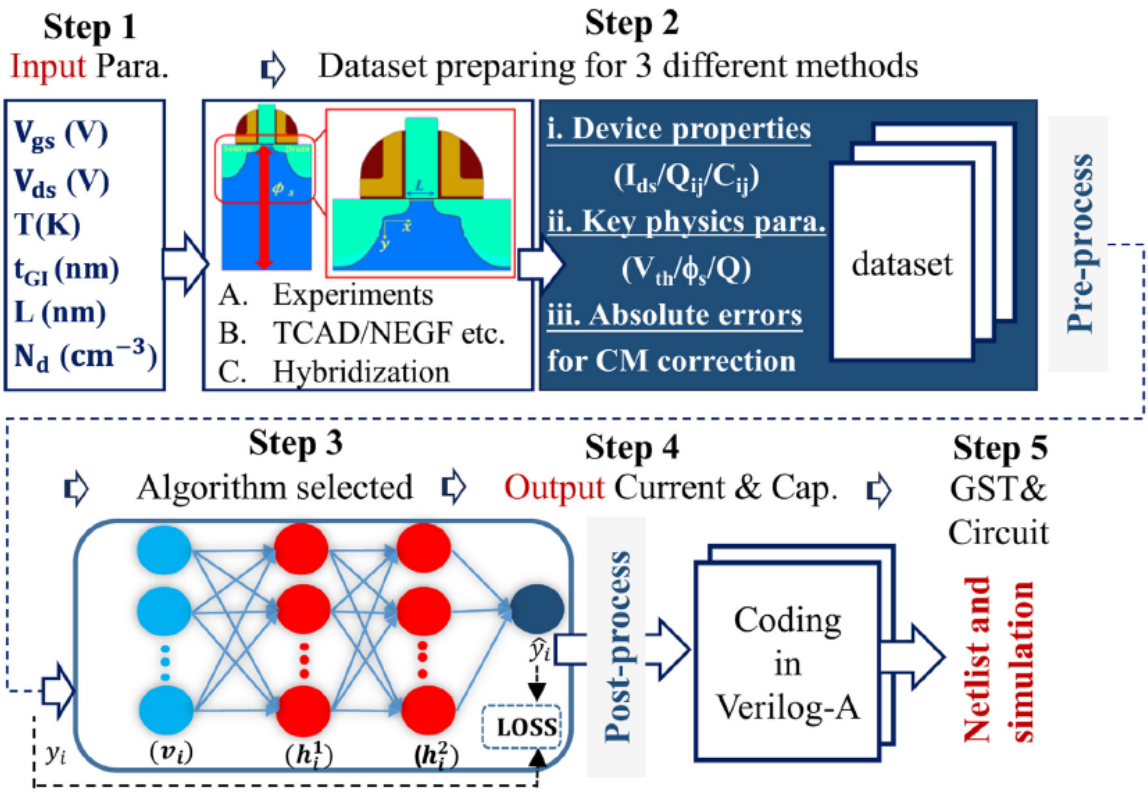


Figure 1.6: The procedure of ML compact model development [2]

scalable approach to compact modeling—paving the way for machine learning-assisted methodologies.[2]

1.3.3 Machine Learning Driven Compact Modeling

To address the growing complexity of nanoscale semiconductor devices and the limitations of traditional compact modeling methods, data-driven approaches—particularly those based on artificial neural networks (ANNs)—have gained substantial momentum in the compact modeling community. ANN-based models provide a flexible alternative by learning the nonlinear mapping between device input parameters and electrical responses directly from data, thereby eliminating the need for hand-crafted physical equations [2, 7]. Although early implementations primarily targeted RF devices, the applicability of ANNs has since broadened to encompass digital and analog domains, owing to their superior fitting accuracy, continuous differentiability, and generalization capacity [2].

ANN methodologies have shown significant promise in reproducing current-voltage (I–V)

and charge-voltage (Q–V) characteristics with high fidelity. Shown in figure 1.6,These models can be trained using either simulation or experimental data and later embedded into SPICE simulators with relatively low computational overhead. When complemented by well-structured loss functions and optimized neural architectures, ANN models exhibit fast SPICE simulation turnaround time (TAT) and demonstrate robust adaptability to process variations—making them particularly suitable for design-technology co-optimization (DTCO) pathfinding efforts [2, 6]. Furthermore, advanced strategies such as automated model generation and model retargeting have enabled efficient reuse of trained ANN architectures across different device types and technology nodes [6].

One of the most promising developments in this area is the advent of physics-inspired neural networks (Pi-NNs), which blend the interpretability of physical modeling with the flexibility of data-driven learning. In the Pi-NN architecture shown in figure 1.7, the drain-source voltage V_{DS} and gate voltage V_{TG} are processed separately, reflecting their distinct physical roles: V_{DS} drives the current through the device, while V_{TG} modulates the channel potential profile.[3] The Pi-NN framework employs two subnetworks: a `tanh`-activated subnet for V_{DS}, which captures both the linear and saturation regions of the I–V curve and enforces zero current at zero bias, and a sigmoid-activated subnet for V_{TG}, which models the exponential and polynomial behavior of subthreshold and above-threshold regimes, respectively. The final output current is computed as the elementwise product of the two subnet outputs, naturally reflecting the interaction between gate control and drain bias.[10] Additionally, cross-layer weighted connections from the `tanh` subnet to the sigmoid subnet account for non-ideal effects such as short-channel modulation, further aligning the model structure with device physics [3]. The Pi-NN has demonstrated strong generalization with minimal parameter complexity, offering a viable compromise between model accuracy and training overhead.

Complementing architectural advances, statistical modeling capabilities have been incorporated into ANN frameworks to handle device variability. Techniques such as backward propagation of variance (BPV) and variational neuron selection enable robust modeling of stochastic process variations and inter-device correlations. These techniques are especially pertinent in modern FinFET and GAA devices, where spatial and structural nonuniformities are prevalent [6]. In early-stage technology development, such variability-

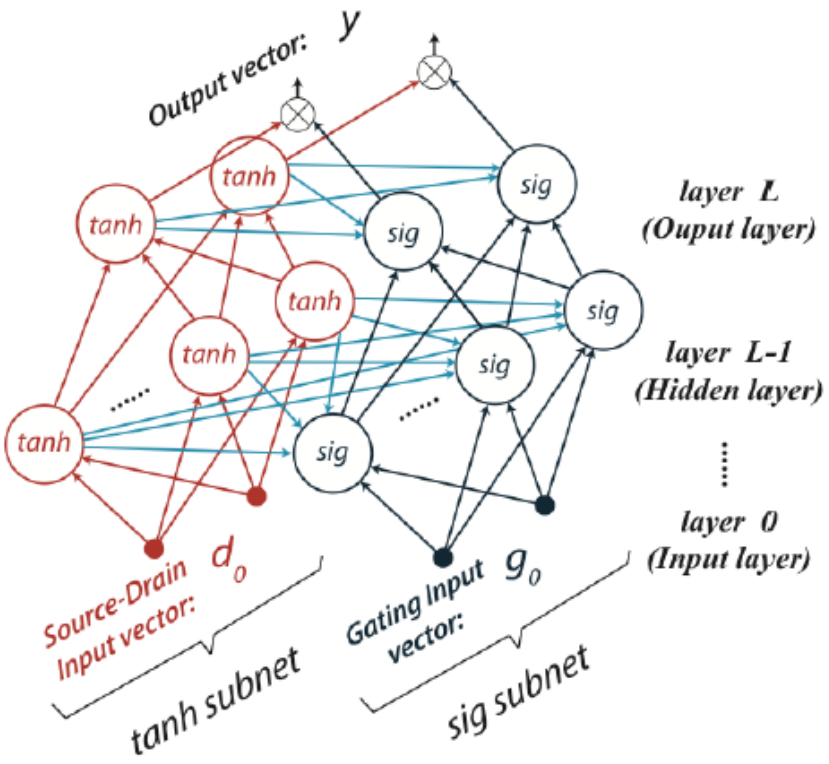


Figure 1.7: Pi-NN Model Structure Clarification [3]

aware modeling approaches provide critical insights into performance envelopes and design margins.

Moreover, the practicality of ANN-based compact models has been enhanced by efficient data sampling strategies. Latin Hypercube Sampling (LHS), for instance, reduces training dataset size while maintaining comprehensive coverage of the input space, which is particularly advantageous in analog design scenarios where high precision is essential. Empirical benchmarks have demonstrated that ANN models outperform traditional models like BSIM in simulating key analog performance metrics such as transconductance and output resistance. This advantage becomes especially evident in complex circuit blocks such as operational amplifiers (OPAMPs) and low-dropout regulators (LDOs) [7].

Despite their promising capabilities, ANN models are often criticized for their lack of physical transparency. Being inherently black-box in nature, they may provide limited interpretability, which can hinder their application in early-stage device design and technology exploration [2]. Additionally, ensuring convergence and generalization of ANN

models in very-large-scale integration (VLSI) simulation environments remains an ongoing research challenge [2]. Hybrid modeling paradigms have been proposed to mitigate these issues. By embedding physical constraints or empirical priors into neural network architectures—as demonstrated by Pi-NNs—researchers aim to retain the predictive power of machine learning while reinforcing physical plausibility and interpretability [7, 3].

Alternative data-driven modeling strategies, such as look-up tables (LUTs), have also been explored. Although LUTs are easy to implement and provide rapid simulation when based on measured data, they suffer from interpolation noise and limited scalability. Furthermore, LUTs do not generalize beyond the pre-specified operating points, restricting their utility in predictive modeling and design space exploration [9]. Compared to LUT-based approaches, ANN models offer superior smoothness, scalability, and adaptability to new data and device topologies.

In summary, the integration of machine learning, particularly ANN-based and physics-informed neural networks, into the compact modeling pipeline offers a transformative solution to the limitations of traditional modeling techniques. These approaches not only enhance prediction accuracy and simulation efficiency but also support the modeling of variability and facilitate the reuse of trained models across technologies. However, further advancements are necessary to address concerns regarding model explainability, physical consistency, and scalability in full-chip design environments. The future of compact modeling is likely to be shaped by hybrid frameworks that synergize data-driven flexibility with the rigor and interpretability of physics-based models.

Chapter 2

Methodology and Results

2.1 Methodology

Compact models can be evaluated using characteristic diagrams such as I_{D}–V_{G}, I_{D}–V_{D}, and other transfer or output characteristics that illustrate the current response under varying bias conditions. In this study, the I_{D}–V_{G} (Id–Vg) curve is selected as the benchmark visualization tool to evaluate the performance of the proposed compact modeling methodology. Specifically, it is used in the final stage to compare the actual and predicted I_{D}–V_{G} curves, allowing for an intuitive assessment of model accuracy.

The methodology consists of two main stages, each implementing its own machine learning (ML) framework. The objective of the first stage is to evaluate the feasibility and accessibility of building a compact model using a simplified TCAD dataset. A basic feedforward artificial neural network, namely a multilayer perceptron (MLE), is used in this stage to fit the reduced dataset and verify the possibility of reconstructing a low-precision compact model. Once this initial stage achieves satisfactory results—producing a basic yet valid I_{D}–V_{G} curve—the second stage is designed to enhance both the model's accuracy and generalization capability.

The second stage builds on the insights from the first framework and adopts a more robust ML pipeline. It incorporates various ANN architectures, including MLE and FatGCN, as well as ensemble learning methods such as Random Forest and XGBoost under bagging and boosting paradigms. This stage utilizes a comprehensive TCAD dataset featuring

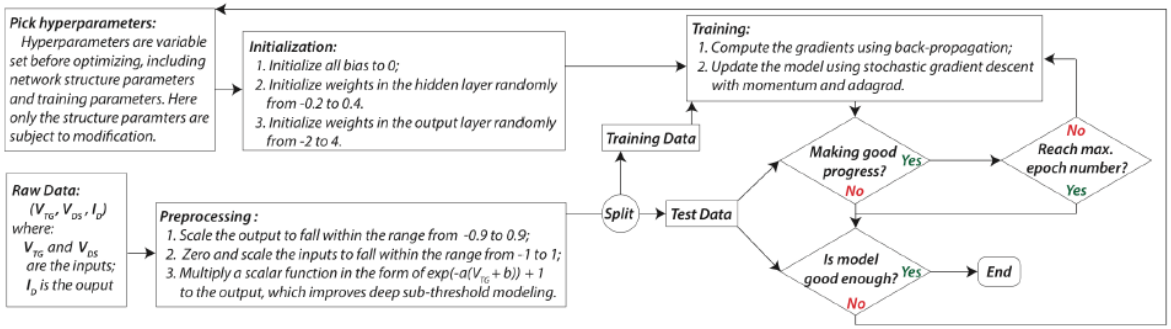


Figure 2.1: Model tuning and optimization procedure for multi-model compact modeling

over ten times more parameters than the first, offering increased complexity and realism for training high-precision compact models. Additionally, a model tuning and parameter optimization framework is established to adjust hyperparameters and model attributes across all employed models, as illustrated in Figure 2.1.

The machine learning models were trained on a Dell G15 5510 laptop with the following hardware specifications:

- **CPU:** 10th Generation Intel® Core™ i7-10870H, featuring 8 cores and 16 threads, with a base clock speed of 2.2 GHz and a maximum turbo frequency of 5.0 GHz.
- **Memory:** 16 GB DDR4 RAM operating at 2933 MHz
- **GPU:** NVIDIA® GeForce® RTX 3060 discrete GPU with 6 GB GDDR6 memory, supporting CUDA acceleration for efficient parallelized model training.
- **Operating System:** Windows 11 Home 64-bit.

2.1.1 ANN (MLE) Model and Basic Demonstration Dataset

The initial modeling experiment utilizes the MLE, a fundamental and widely-used ANN model, to serve as a performance baseline.

Data Selection and Preprocessing

The initial TCAD simulations generated four datasets: nFET I_D–V_G, nFET I_D–V_D, pFET I_D–V_G, and pFET I_D–V_D. These are summarized in Table 2.1. After evaluating the datasets, the nFET I_D–V_G dataset is selected due to its clear and representative transfer characteristics, which are essential for validating the baseline compact model.

Dataset Type	Description
nFET Id–Vg	Drain current vs. gate voltage (n-type)
nFET Id–Vd	Drain current vs. drain voltage (n-type)
pFET Id–Vg	Drain current vs. gate voltage (p-type)
pFET Id–Vd	Drain current vs. drain voltage (p-type)

Table 2.1: Overview of extracted TCAD simulation datasets

As shown in Figure 2.2, the selected dataset contains 2,476 data points and features three input parameters: V_{DS}, V_{GS}, and L_G (channel length). The target output is I_D, which the model aims to predict. Although the dataset is relatively small in both size and feature dimension, it is sufficient for validating the feasibility of compact modeling via machine learning.

Additionally, the dataset including the second dataset is not randomly mixed but segmented by different device configurations. Since the goal is to obtain meaningful Id–Vg diagrams, each segment of data is pre-structured such that the other semiconductor parameters remain constant, while only the gate voltage (V_G) is varied. This ensures that the output current (I_D) can be visualized cleanly and continuously across different gate bias points for each device instance, enabling consistent and interpretable performance evaluation.

To improve training performance and model convergence, the output current I_D is logarithmically scaled. The transformation is defined as follows: Logarithmic transformation of drain current Y = -0.1 * torch.where(Y ¡ 0, torch.log(-Y), torch.log(Y))

vds	vgs	lg	ids
0.1	3	0.25	2.00E-05
0.1	2.9805	0.25	2.00E-05
0.1	2.9605	0.25	2.01E-05
0.1	2.941	0.25	2.00E-05
0.1	2.921	0.25	1.98E-05
0.1	2.9015	0.25	1.98E-05
0.1	2.8815	0.25	1.97E-05
0.1	2.862	0.25	1.97E-05
0.1	2.842	0.25	1.96E-05
0.1	2.8225	0.25	1.96E-05
0.1	2.8025	0.25	1.96E-05
0.1	2.783	0.25	1.95E-05
0.1	2.763	0.25	1.94E-05
0.1	2.7435	0.25	1.93E-05
0.1	2.7235	0.25	1.92E-05
0.1	2.704	0.25	1.93E-05
0.1	2.684	0.25	1.93E-05
0.1	2.6645	0.25	1.92E-05
0.1	2.6445	0.25	1.93E-05
0.1	2.625	0.25	1.92E-05
0.1	2.6055	0.25	1.91E-05
0.1	2.5855	0.25	1.91E-05
0.1	2.566	0.25	1.90E-05
0.1	2.546	0.25	1.89E-05
0.1	2.5265	0.25	1.89E-05
0.1	2.5065	0.25	1.89E-05
0.1	2.487	0.25	1.89E-05
0.1	2.467	0.25	1.88E-05
0.1	2.4475	0.25	1.88E-05
0.1	2.4275	0.25	1.86E-05
0.1	2.408	0.25	1.85E-05

Figure 2.2: nFET I_{D}–V_{G} dataset used in the first-stage modeling

Model Establishment

The compact model in this study is constructed based on a feedforward artificial neural network (ANN) architecture, as illustrated in Figure 2.3. The model consists of an input layer, multiple hidden layers, and an output layer, each composed of interconnected neurons. This structure enables the model to approximate complex nonlinear relationships between device parameters and the resulting electrical characteristics, such as the drain current I_{D}.

The architecture follows a layer-wise formulation:

- The input vector $\boldsymbol{x}_0$ represents the physical parameters of the semiconductor device, such as V_{DS}, V_{GS}, and L_{G}.

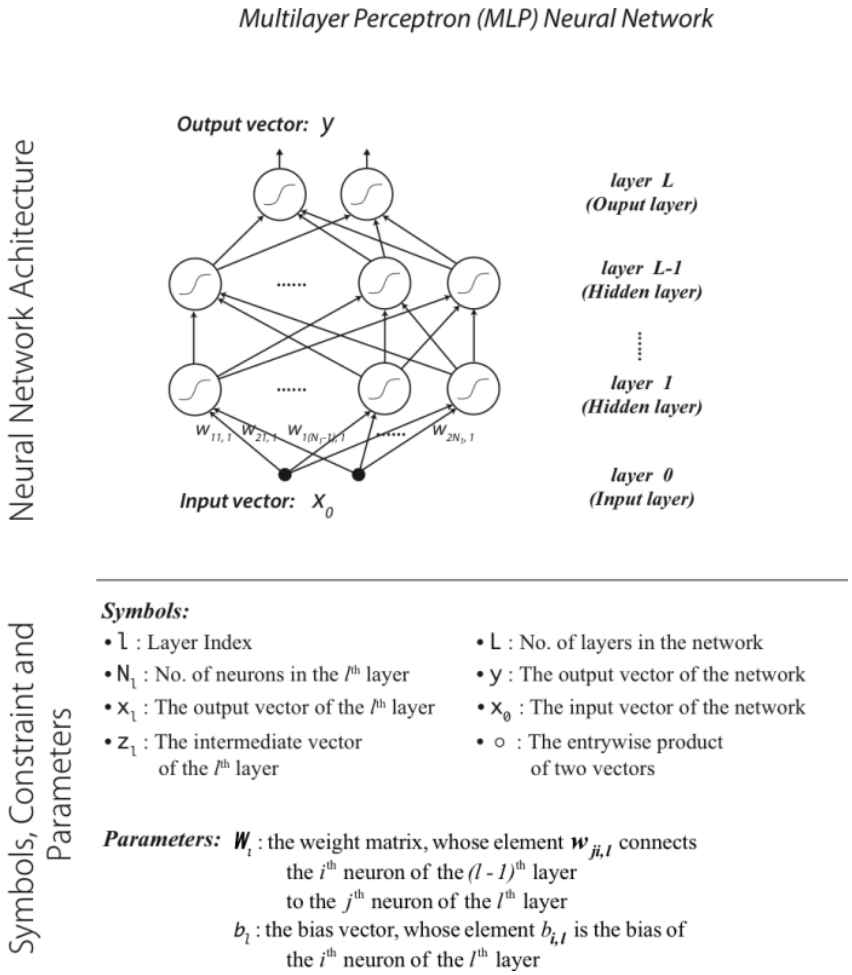


Figure 2.3: General Neural Network Architecture and Symbol Definitions

- Each hidden layer $l \in \{1, \ldots, L-1\}$ transforms its input through a weighted sum and non-linear activation function. The transformation for the ith neuron in layer l is given by:

$$z_i^l = \sum_j W_{ij}^l x_j^{l-1} + b_i^l, \tag{2.1}$$

where W_{ij}^l denotes the weight connecting the jth neuron of the $(l-1)^{\text{th}}$ layer to the ith neuron of the lth layer, and b_i^l is the associated bias.

- The output of the neuron is obtained by applying a non-linear activation function $\phi(\cdot)$ to z_i^l:

$$x_i^l = \phi(z_i^l) \tag{2.2}$$

- The final output vector $\boldsymbol{y}$ is obtained at layer L, which predicts the drain current I_{D} for given input conditions.

The model training involves optimizing the weights and biases across all layers to minimize the loss function, typically the Mean Squared Error (MSE) between predicted and true I_{D} values. Backpropagation with gradient descent (or its variants such as Adam) is used for optimization.

This formulation provides a flexible and differentiable framework capable of modeling highly nonlinear behavior observed in advanced semiconductor devices.

Tuning Procedure

The tuning procedure forms a critical part of the model development pipeline, serving to optimize the performance of the neural network through structured preprocessing, initialization, training, and iterative evaluation. This process, illustrated in Figure 2.1 and Figure 2.5, begins with the definition of hyperparameters and structural choices for the model. These include the number of layers, the number of neurons per layer, and the choice of optimization algorithms. Prior to training, the network is initialized with biases set to zero, while the weights are randomly sampled from specified ranges: hidden layer weights from -0.2 to 0.4, and output layer weights from -2 to 4. This initialization strategy helps avoid symmetry and improves convergence behavior in early training stages.

Raw TCAD simulation data, comprising tuples of input voltages V_{DS}, V_{GS}, gate length L_{G}, and the corresponding drain current I_{D}, is preprocessed to improve numerical stability and learning efficiency. The output I_{D} is scaled to fall within the range $[-0.9, 0.9]$, while the input voltages are normalized to the range $[-1, 1]$. To further enhance modeling accuracy in the sub-threshold region, especially for deep learning applications, a nonlinear scalar function of the form $\exp(-\alpha(V_{\mathrm{GS}} + b)) + 1$ is applied to the output values.

The core model utilized in this stage is a Multi-Layer Perceptron (MLE), consisting of an input layer with three neurons (corresponding to V_{DS}, V_{GS}, and L_{G}), two hidden layers with 30 and 10 neurons respectively using ReLU activation functions, and a final output layer producing a single prediction for I_{D}. The model is implemented using PyTorch and trained using the Adam optimizer with a learning rate of 0.0002. The loss function

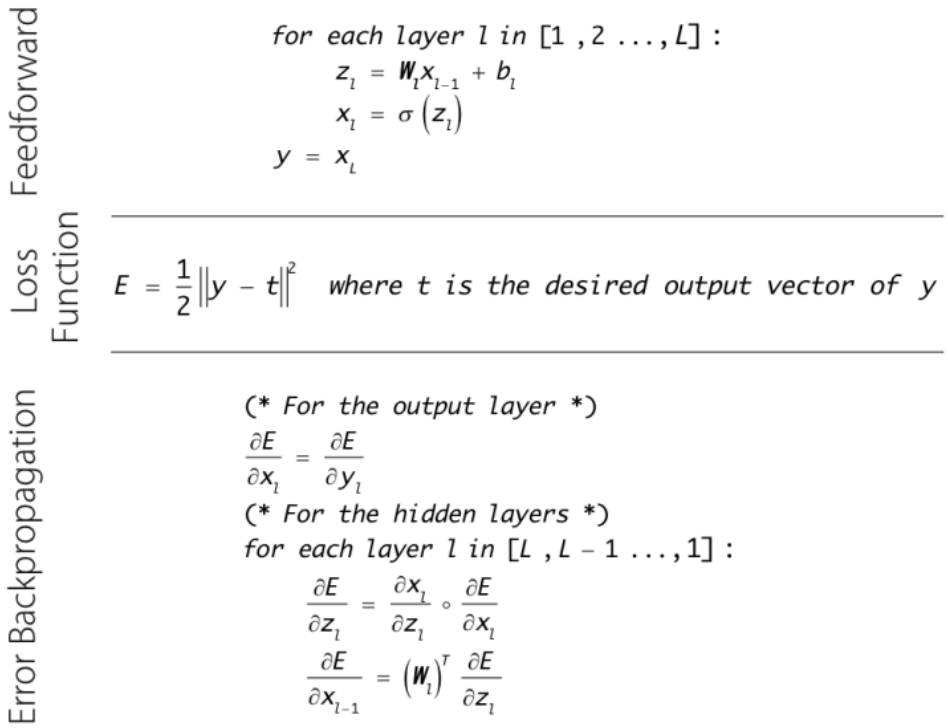


Figure 2.4: MLE Model Training procedure & Loss calculation

employed is the root mean square error (RMSE), which provides a direct measure of the deviation between predicted and actual current values.

During training, the dataset is split into training and testing subsets. For each epoch, the model undergoes a forward pass, loss computation, and parameter update through backpropagation. Validation is conducted after every epoch using the test set, with validation RMSE recorded to monitor the model's generalization ability. The training continues for up to 100 epochs or until the model's performance plateaus, based on the criteria defined in the flowchart. Each epoch's result is saved, and the best-performing model is retained for future data prediction.

2.1.2 ANN, Bagging/Boosting Models and Real-World Dataset

Following the initial attempts with various machine learning models, the simulation results for the compact model still failed to achieve satisfactory accuracy, as observed from the Id-Vg visualization diagrams. A key reason for this suboptimal performance is attributed to the limited set of input parameters. In the previous stage, only three

parameters—L_g, V_{GS}, and V_{DS}—were used to predict I_{D}, which proved insufficient to capture the complex behavior of the device. In reality, many other device parameters significantly influence the Id-Vg characteristics. Neglecting these factors led to incomplete modeling of the device's behavior.

To address this limitation, a more comprehensive TCAD simulation dataset was utilized for further training. This dataset includes a wider range of physical and geometrical parameters, allowing for a more precise and representative compact model. Additionally, the performance of the compact model is highly dependent on the choice of machine learning model and its configuration, such as whether an Artificial Neural Network (ANN) or ensemble methods like Random Forest and XGBoost are employed [2, 3, 7].

Data Preparation

The raw TCAD simulation output was first exported and then converted into a structured Excel format suitable for machine learning workflows. As shown in Figure 2.5, the updated dataset features seven key input parameters, including gate length (L_g), channel length (L_{ch}), oxide thickness (T_{ox}), among others, offering a richer and more diverse input space. The total size of the dataset is approximately 75,000 samples, a significant increase compared to the initial simple dataset.

Using a larger and more comprehensive dataset offers several advantages. It enables better coverage of the device's operational space, improves the model's generalization ability, and reduces the risk of overfitting [11]. Moreover, the inclusion of more physical parameters allows the model to learn complex nonlinear interactions, which are essential for accurately predicting I_{D} across varying bias conditions and device structures.

Feature Selection

Before formally preprocessing the data for training, a feature selection step was conducted to verify whether each input parameter positively contributes to the simulation performance. The primary goal of this process is to ensure that only meaningful features are used, thereby improving the model's accuracy and efficiency while reducing potential

	A	B	C	D	E	F	G	H
1	Lg	Lch	Tox	Nsub	PeakV	Vds	Vgs	Id
2	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-1	3.22E-16
3	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.97475	3.22E-16
4	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.94949	3.21E-16
5	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.92424	3.23E-16
6	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.89899	3.23E-16
7	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.87374	3.24E-16
8	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.84848	3.25E-16
9	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.82323	3.26E-16
10	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.79798	3.27E-16
11	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.77273	3.29E-16
12	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.74747	3.31E-16
13	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.72222	3.36E-16
14	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.69697	3.41E-16
15	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.67172	3.60E-16
16	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.64646	3.83E-16
17	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.62121	4.07E-16
18	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.59596	5.51E-16
19	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.57071	7.48E-16
20	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.54545	1.02E-15
21	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.5202	1.53E-15
22	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.49495	2.77E-15
23	0.1	0.2	0.008	1.00E+18	6.00E+20	0.2	-0.4697	5.57E-15

Figure 2.5: TCAD Simulation Dataset Preparation

bias introduced by irrelevant inputs.

The feature selection procedure follows the methodology described in [13]. Feature importance is a critical step for understanding which inputs significantly influence model predictions. It also helps prevent the negative effects that irrelevant features can have, such as model overfitting or distorted results. Among various techniques, the Permutation Feature Importance method was selected due to its simplicity and effectiveness. In this technique, the importance of each feature is determined by measuring the increase or decrease in model error when the feature's values are randomly shuffled. A large change in error implies that the feature is important for the model's predictive capability.

In this study, feature importance was assessed using a Random Forest Regressor, with Mean Squared Error (MSE) as the evaluation metric, rather than F1-score which is more typical for classification tasks. The dataset was first split into training and testing sets, and a Random Forest model was trained on the training data. After calculating the baseline MSE on the test set, the feature importance scores were derived based on how

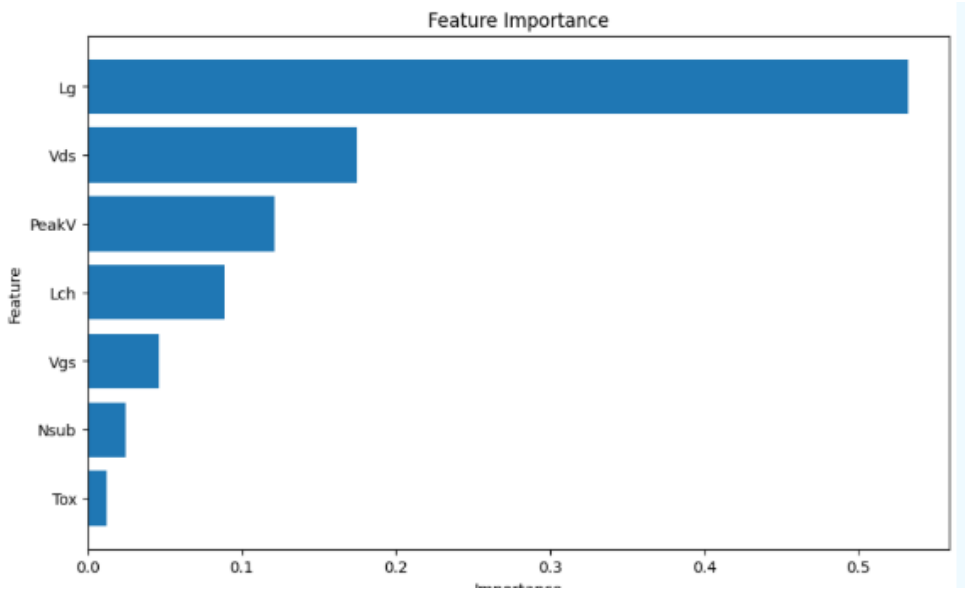


Figure 2.6: Feature Importance Analysis

much each feature influenced the model's performance. The full implementation of this procedure is shown in appendix.

The resulting feature importance values are presented in Figure 2.6. It can be observed that all seven parameters demonstrate a positive contribution to the model's performance, thus confirming that each feature is relevant and should be retained for further training and modeling.

Data Pre-Process

Before training the models, an essential data preprocessing step was conducted to ensure the quality, consistency, and usability of the dataset. The preprocessing procedure involved three major aspects: data cleaning, data normalization, and data sampling, each described in detail below.

Data Cleaning The raw TCAD simulation data was first exported and organized into a structured DataFrame, where seven device parameters—L_g, L_{ch}, T_{ox}, N_{sub}, V_{peak}, V_{ds}, and V_{gs}—were selected as input features, and I_d (drain current) was designated as the output target. During data cleaning, missing values, if any, were checked and handled appropriately to ensure no invalid entries remained. Additionally, the correctness of data

types was verified, and entries with obviously erroneous physical values were removed.

Data Normalization To improve the model's convergence and training stability, a two-stage normalization was applied to the data.

First, a logarithmic transformation was used to compress the wide dynamic range of the drain current I_d. This transformation can be expressed as:

$$Y_{\log} = \log_{10}(Y)$$

where Y represents the original drain current values. Logarithmic scaling is critical for semiconductor device modeling because the current values often span multiple orders of magnitude, especially in the subthreshold and high-field regions.

After applying the logarithmic transformation, both the input features and the output target were further normalized using the `StandardScaler` method. `StandardScaler` standardizes each feature by removing the mean and scaling to unit variance, following the formula:

$$X_{\text{norm}} = \frac{X - \mu_X}{\sigma_X} \quad \text{and} \quad Y_{\text{norm}} = \frac{Y_{\log} - \mu_Y}{\sigma_Y}$$

where μ_X and μ_Y are the mean values, and σ_X and σ_Y are the standard deviations of the input features and output targets, respectively.

This standardization ensures that each feature contributes equally to the model training process and prevents features with larger numerical ranges from dominating the learning algorithm.

Data Sampling An important aspect of data preparation involved the method used to split the dataset into training and testing subsets. A random selection method is commonly used in general machine learning tasks; however, in this context, random sampling would disrupt the sequential order of the simulation data. Since the visualization

of Id–Vg curves and other device characteristics depends heavily on maintaining the correct parameter progression, preserving the order was critical.

Therefore, instead of using purely random sampling, a sequential slicing method was implemented. The first 80% of the data samples were allocated for training, while the remaining 20% were reserved for testing. Mathematically, the split was performed as:

$$\text{Training Set}: \quad X_{\text{train}}, Y_{\text{train}} = X[0:0.8N], Y[0:0.8N]$$

$$\text{Testing Set}: \quad X_{\text{test}}, Y_{\text{test}} = X[0.8N:N], Y[0.8N:N]$$

where N is the total number of samples. This approach ensures that the inherent structure and progression of the data are preserved, allowing for accurate visualization and avoiding artificial discontinuities that would mislead model evaluation and interpretation.

Through this careful preprocessing pipeline, the dataset was made ready for subsequent model training while ensuring both numerical stability and correct physical interpretation.

Model Establishing & Training

This subsubsection describes the establishment and training of three modelling paradigms: a baseline multilayer perceptron (MLE), a fully-connected graph-inspired network (FatGCN), and two ensemble methods (Random Forest and XGBoost). The full TCAD dataset was partitioned sequentially into 80% for training and 20% for testing to preserve the inherent order of device simulation data. The detailed structure of models are listed in Appendix A.

ANN Models: MLE and FatGCN The MLE model was established in the previous framework and, due to its relatively simple architecture comprising only 325 parameters, a FatGCN model was additionally implemented to achieve higher predictive precision. Both models belong to the class of Artificial Neural Networks (ANNs) and share similar foundational components, as illustrated in Figure A.1. However, the FatGCN architecture extends beyond the basic MLE structure by incorporating additional layers and wider

hidden dimensions, enhancing its capacity to model complex nonlinear interactions among the seven input features.

In the implemented FatGCN model, the network consists of an input layer with dimension d_{in}, followed by two fully connected hidden layers, each with d_{hid} nodes, and an output layer generating a single scalar value corresponding to the drain current I_{D}. ReLU activation functions are applied between all layers. The model training was performed over 100 epochs using the Adam optimizer, minimizing the mean squared error (MSE) loss. During each epoch, forward propagation was executed over mini-batches, followed by backpropagation and parameter updates. Validation MSE was evaluated after each epoch using the held-out test set, and the model parameters corresponding to the lowest validation MSE were retained.

The structural differences between the MLE and FatGCN models are summarized in Table 2.2. Notably, FatGCN incorporates three GCN layers, mean pooling, LayerNorm normalization, and a significantly larger parameter count (0.154M compared to MLE's 325 parameters), which collectively contribute to its improved modeling capabilities.

	FatGCN	**MLE**
GCN/GAT layer	3	3
Hidden layer (node)	1(64), 1(128), 1(256)	1(30), 1(10), 1(11)
Pooling layer	mean	None
MLE layer	4	1
Hidden layer (node)	1(256), 1(128), 1(64), 1(1)	1(30), 1(10), 1(1)
Total layer	7	3
Normalization	Layernorm	Layernorm
Parameters	154,000	325

Table 2.2: Comparison of FatGCN and MLE architectures based on implemented models.

Ensemble Models: Random Forest and XGBoost Bagging and boosting are two ensemble learning strategies that enhance model generalization through distinct mechanisms. Bagging (Bootstrap Aggregating) constructs multiple independent models trained on random subsets of the data, then aggregates their predictions to reduce variance. Boosting, by contrast, sequentially fits models, with each new model focusing on correcting the errors of its predecessors, thereby reducing both bias and variance.

The Random Forest regressor used in this work follows the bagging principle, employing an ensemble of decision trees. Each tree is trained on a bootstrap sample of the dataset, with randomized feature selection at each split to encourage model diversity. Several hyperparameters were systematically tuned via grid search to optimize model performance. The number of estimators, set between 50 and 100, controls the size of the forest; a larger number generally improves performance at the cost of higher computational demand. Maximum tree depth, varied from 10 to 20, limits the depth to which each tree can grow, thereby managing model complexity and mitigating overfitting. Additionally, the minimum number of samples required to split a node (2, 5, or 10) and the minimum number of samples required at a leaf node (1, 2, or 5) were tuned to balance tree size and predictive robustness. Model evaluation was conducted based on mean absolute error (MAE) on a validation set, and the hyperparameter configuration yielding the lowest MAE was selected for final training.

The XGBoost regressor was designed according to the boosting strategy. In contrast to Random Forest, XGBoost builds trees sequentially, with each new tree trained to predict the residual errors of the current ensemble. The model hyperparameters were tuned to optimize both learning efficiency and generalization performance. The number of boosting rounds, varied from 100 to 300, determines the number of additive models contributing to the final prediction. Maximum tree depth, restricted between 3 and 7, controls the complexity of each individual tree to prevent overfitting in a boosting context. The learning rate, explored within the range of 0.01 to 0.2, governs the contribution of each new tree, allowing fine-grained updates that stabilize the training process. Furthermore, the subsample ratio (0.8 or 1.0) specifies the fraction of training data used per boosting round to inject randomness and prevent overfitting, while the column subsampling rate (0.8 or 1.0) reduces model complexity by randomly selecting feature subsets at each tree split. Grid search with five-fold cross-validation was applied to identify the hyperparameter combination minimizing the validation MAE, and the final model was retrained using the full training dataset.

Compared to traditional artificial neural network (ANN) models such as the multilayer perceptron (MLE) and FatGCN architectures discussed earlier, these ensemble models differ fundamentally in both structure and optimization. ANNs consist of multiple layers

of neurons with nonlinear activation functions, trained end-to-end using gradient-based optimization, typically requiring feature normalization and careful regularization. In contrast, ensemble tree models aggregate multiple shallow decision trees, capturing nonlinearity and complex feature interactions inherently without requiring data preprocessing or differentiable operations. These complementary strengths have been widely recognized in machine learning literature [14, 15], making ensemble methods attractive candidates for modeling complex physical behaviors such as the $I_D - V_G$ relationship in advanced semiconductor devices.

	Random Forest	**XGBoost**
Type	Bagging	Boosting
Base Learner	Decision Tree	Decision Tree
Number of Estimators	50–100	100–300
Maximum Depth	10–20	3–7
Learning Rate	None	0.01–0.2
Sampling	Bootstrap sampling of data	Sequential sampling with boosting
Feature Subsampling	Yes (random subsets)	Yes (colsample_bytree parameter)
Parameter Tuning	Grid Search	Grid Search
Typical Application	Reducing variance	Reducing bias and variance

Table 2.3: Model Structure of Random Forest and XGBoost models.

2.2 Results

The experimental results confirm that both machine learning frameworks successfully constructed compact models capable of predicting device behavior with high precision. To assess the model accuracy, the mean squared error (MSE) of the validation set was used as the evaluation metric, capturing the deviation between the original I_D values and the predicted values across varying V_G conditions.

For the first framework, the training progression is illustrated in Figure 2.7. The MSE steadily decreases as training proceeds, reaching a stable plateau within only 10 epochs. The minimum MSE achieved was approximately 0.0012, indicating a high degree of precision for the initial compact model. The smooth and rapid convergence of the loss function demonstrates the model's ability to effectively learn the underlying physics from

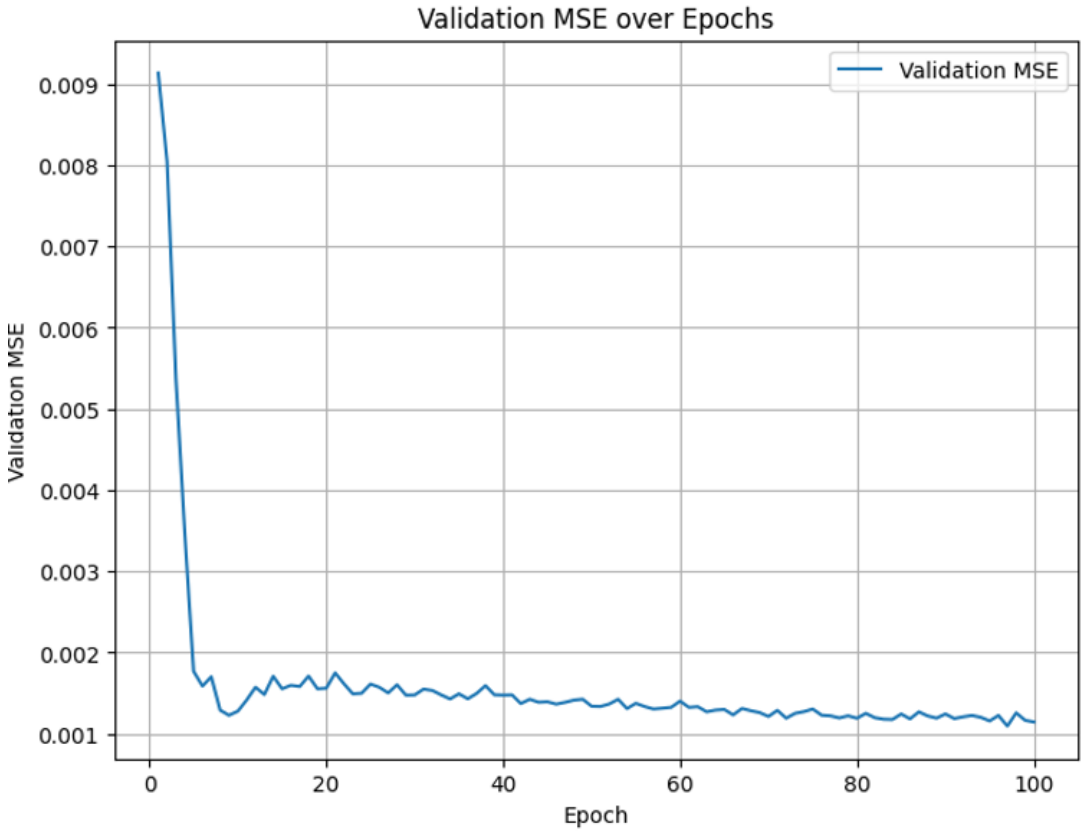


Figure 2.7: MSE descent curve for the basic MLE framework.

the simplified dataset.

For the second framework, which introduced more complex models and datasets, the performance comparison among different models is presented in Figure 2.8. After extensive hyperparameter tuning, including model depth and learning rates, the MLE and FatGCN models demonstrated a more stable convergence of the mean absolute error (MAE). It should be noted that, following conventions in recent literature [4], MAE rather than MSE was selected as the evaluation metric for the second framework. Unlike MSE, MAE does not square the residuals, leading to more pronounced fluctuations during training but offering a more direct measure of prediction error.

Among the tested models, FatGCN exhibited the worst performance, likely due to incomplete convergence; with 1,234,567 trainable parameters, the model's training could benefit from further hyperparameter optimization and access to more powerful computational resources. In contrast, the basic MLE model achieved moderate performance with only 561 parameters, successfully reaching an ideal descent curve. The Random

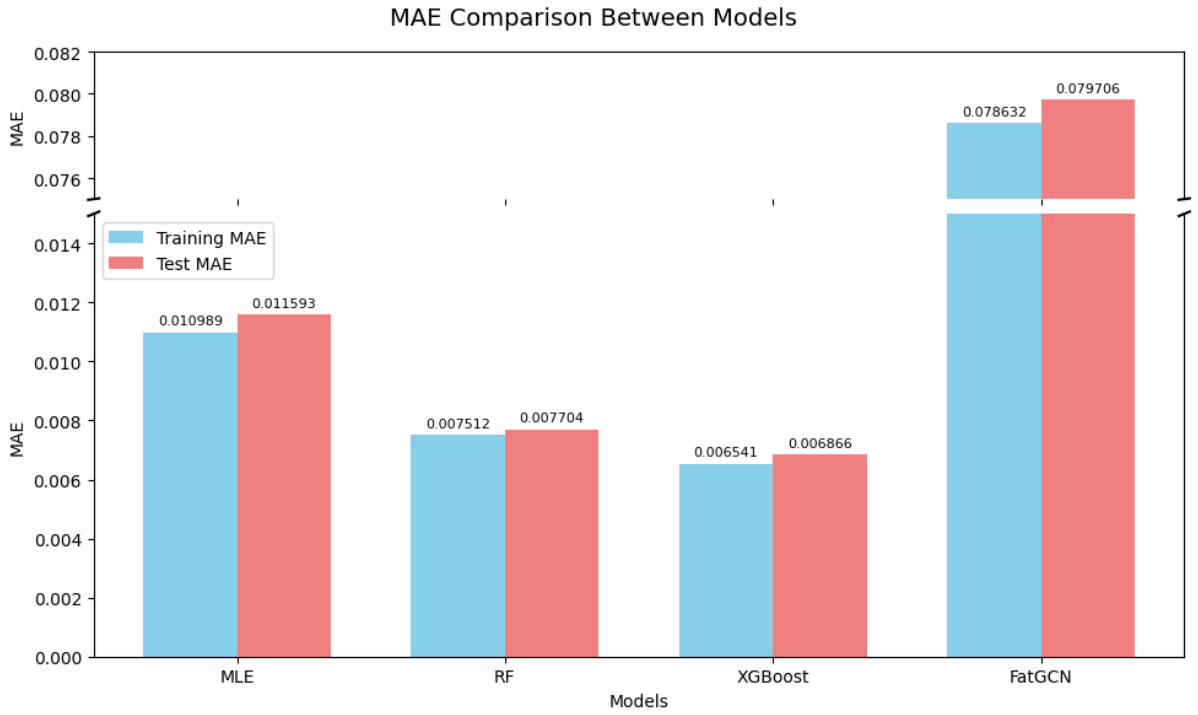
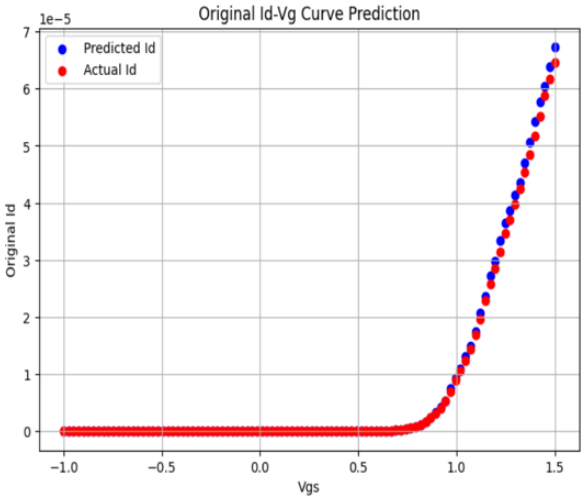


Figure 2.8: Comparison of MAE convergence among different models in the second framework.

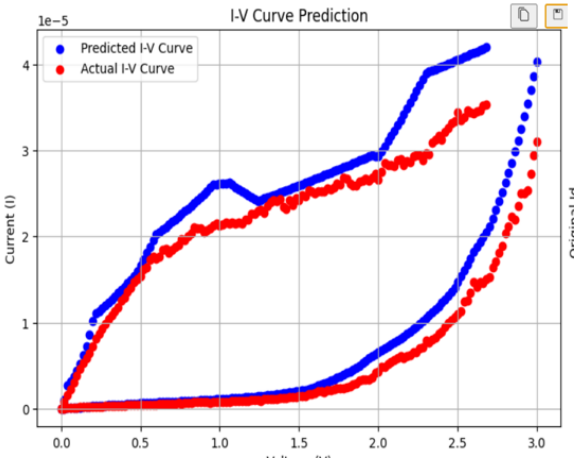
Forest and XGBoost models delivered the best results, attaining a minimum MAE of approximately 0.0068.

While the first framework achieved a lower training error (MSE) compared to the second framework (MAE), it is important to emphasize that the evaluation criteria and datasets differ significantly between the two frameworks. The simpler dataset and normalization applied in the first framework make direct comparisons of MSE and MAE between frameworks inappropriate. Instead, the advancements achieved through the second framework are better illustrated through direct visualization of the $I_{\mathrm{D}} - V_{\mathrm{G}}$ characteristics, highlighting the effectiveness of the improved models.

These results indicate that both frameworks successfully achieved the goal of compact model prediction, with the second framework attaining higher precision through model tuning and hyperparameter optimization. This outcome supports the hypothesis that applying realistic datasets in conjunction with advanced models leads to more accurate and robust compact modeling. However, the performance of such advanced models strongly depends on appropriate tuning and adjustment procedures to unlock their full potential. Additionally, the adoption of machine learning methodologies significantly reduces the



(a) Predicted $I_D - V_G$ curve for FatGCN model.



(b) Predicted $I_D - V_G$ curve for initial MLE model.

Figure 2.9: Comparison of $I_D - V_G$ prediction accuracy between FatGCN and MLE models.

need for costly physical experiments and semiconductor material consumption, contributing to a more sustainable and environmentally friendly design process with zero direct pollution.

2.3 Evaluation

To provide a visual assessment of model precision, the predicted $I_D - V_G$ curves for the most accurate models in each framework are presented in Figure 2.9. Both diagrams are plotted on the same scale (current values in the order of 10^{-5}) to ensure a fair comparison.

As shown in Figure 2.9a, the FatGCN model from the second framework exhibits predictions closely aligned with the true device characteristics, with minimal deviation across the entire V_G range. In contrast, the basic MLE model shown in Figure 2.9b demonstrates larger discrepancies, particularly at lower gate voltages. These results not only validate the effectiveness of the second framework in improving model fidelity but also highlight the readiness of the compact models for practical applications in device simulation and industrial modeling tasks.

Chapter 3

Conclusion and Future Work

3.1 Conclusion

This project explores a wide range of applications of compact model prediction and investigates the historical evolution of compact modeling techniques, starting from traditional physics-based methods to machine learning-driven approaches, including artificial neural networks (ANNs), bagging-based Random Forests, and boosting-based XGBoost models.

After thorough investigation, a prototype framework was established, as shown in Figure 1.3, to guide the creation of detailed compact model prediction frameworks through Data Selection, Model Establishment, and Model Tuning stages. Building upon the limitations observed in the initial experiments, a second, more advanced framework was developed, utilizing a realistic TCAD simulation dataset and incorporating additional stages such as Data Preparation, Feature Selection, and a more comprehensive model tuning process.

The prediction of compact models for unknown devices and their corresponding $I_{\mathrm{D}} - V_{\mathrm{G}}$ evaluations was successfully completed. The evaluation results clearly demonstrate that the second framework achieved significantly higher precision compared to the initial one. This validates the original hypothesis that using a realistic dataset and more advanced machine learning models would yield higher prediction accuracy. Furthermore, the experiments reveal that achieving optimal model precision requires extended training and careful parameter tuning.

A limitation of this study is that, although the predicted compact models demonstrated relatively low loss values (best MAE of 0.0068 after normalization), there is still noticeable room for improvement, especially in achieving industrial-grade compact model precision. Additionally, further assessment is needed to map the achieved MAE levels to specific industrial applications.

The significance of these results lies in their contribution to Design-Technology Co-Optimization (DTCO) frameworks. In such workflows, compact models act as critical bridges between semiconductor fabrication and circuit design, enabling efficient simulation for power, performance, area, and cost (PPAC) optimization [2]. As illustrated in Figure 1.3, TCAD-to-SPICE methodologies rely on highly accurate compact models that incorporate real-world phenomena like parasitic effects, hot carrier degradation (HCD), and quantum effects [2]. Therefore, the development of precise ML-driven compact models, as demonstrated in this study, supports the industrial move toward faster, more efficient DTCO processes.

Although the current models achieve relatively close predictions to the actual device behavior, further efforts are needed to both lower prediction loss and critically assess whether the models meet the thresholds necessary for industrial deployment.

3.2 Future Work

3.2.1 FatGCN Further Tuning

To achieve compact models with even higher precision, further tuning of the FatGCN model is a key direction. Currently, the FatGCN model demonstrated the lowest precision among the four machine learning models despite having the most trainable parameters (approximately 154,000). This performance contrasts with previous studies, where FatGCN achieved superior accuracy compared to Random Forest and basic ANN models [4]. The discrepancy may result from factors such as insufficient training data, limited training epochs, suboptimal parameterization, or constrained GPU performance.

To address these issues, future work will focus on utilizing the full second dataset and

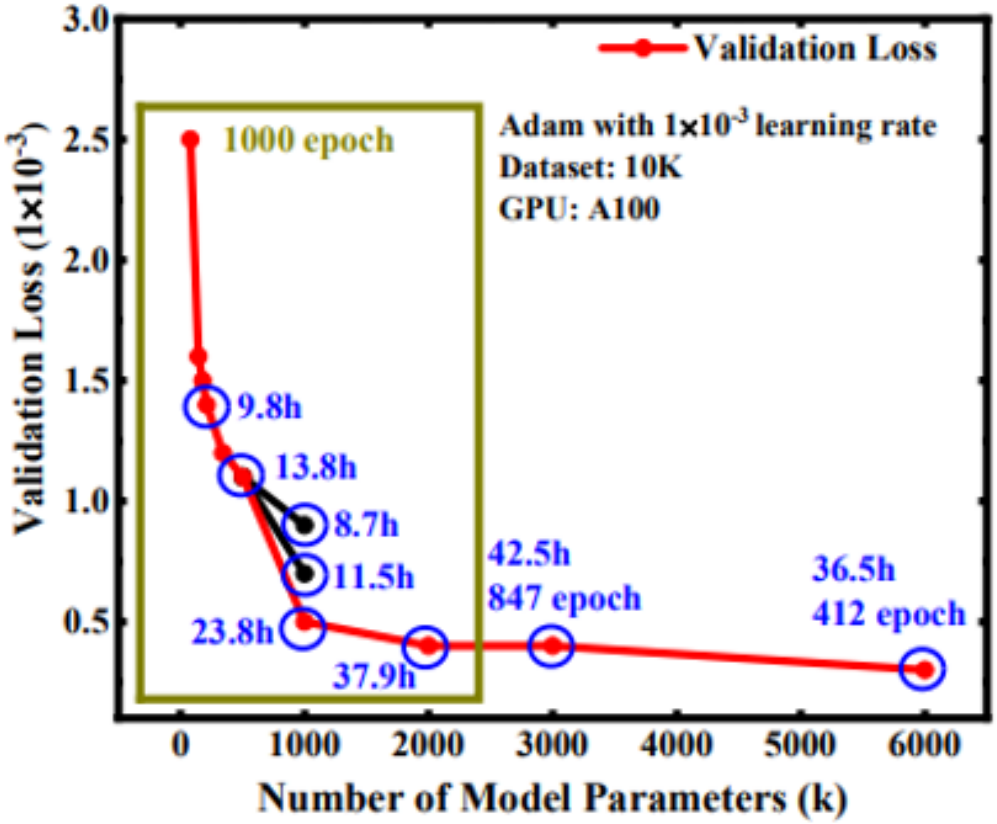


Figure 3.1: Referenced FatGCN model tuning procedure [4]

leveraging high-performance computing (HPC) resources by applying for access to university HPC facilities. Additionally, tuning the number of parameters in relation to observed training loss and adjusting model complexity will be essential steps. A reference for FatGCN tuning procedures is shown in Figure 3.1.

3.2.2 Assessment for Current Training Loss and Criteria

Another critical area for future work involves establishing a standardized assessment of the training loss relative to industrial application thresholds. Currently, while low MAE values are achieved, the realistic meaning of such metrics must be justified—industrial standards often require MAE below 0.02 for practical deployment. Therefore, a systematic literature review of semiconductor manufacturing and compact modeling studies will be conducted to establish clear benchmarks.

Defining these thresholds will enable a more precise evaluation of whether the compact

models developed here are suitable for use in DTCO workflows, SPICE simulations, or other industrial contexts. This work is achievable within reasonable cost and time constraints through continued research and academic collaborations.

Bibliography

[1] J. Kusters, D. Bhattacharjee, and J. Bish. (2024) 2024 semiconductor industry outlook. Deloitte Insights. United States/Netherlands. [Online]. Available: https://www2.deloitte.com/us/en/insights/industry/technology/technology-media-telecom-outlooks/semiconductor-industry-outlook.html

[2] X. Li, Z. Wu, G. Rzepa, M. Karner, H. Xu, Z. Wu, W. Wang, G. Yang, Q. Luo, L. Wang, and L. Li, "Overview of emerging semiconductor device model methodologies: From device physics to machine learning engines," *Fundamental Research*, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667325824000323

[3] M. Li, O. İrsoy, C. Cardie, and H. G. Xing, "Physics-inspired neural networks for efficient device compact modeling," *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 2, pp. 44–49, 2016.

[4] G. Fan, T. Ma, X. Sun, L. Shao, and K. L. Low, "Graph attention network-based unified tcad modeling enabling fast design technology co-optimization through transfer learning," *IEEE Transactions on Electron Devices*, vol. 72, no. 1, pp. 474–481, 2025.

[5] H. H. Radamson, H. Zhu, Z. Wu, X. He, H. Lin, J. Liu, J. Xiang, Z. Kong, W. Xiong, J. Li, H. Cui, J. Gao, H. Yang, Y. Du, B. Xu, B. Li, X. Zhao, J. Yu, Y. Dong, and G. Wang, "State of the art and future perspectives in advanced cmos technology," *Nanomaterials*, vol. 10, no. 8, 2020. [Online]. Available: https://www.mdpi.com/2079-4991/10/8/1555

[6] W. Dai *et al.*, "Statistical compact modeling with artificial neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 5156–5160, 2023.

[7] J. Wei, H. Wang, T. Zhao, Y.-L. Jiang, and J. Wan, "A new compact mosfet model based on artificial neural network with unique data preprocessing and sampling techniques," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 4, pp. 1250–1254, 2023.

[8] C.-T. Tung, M.-Y. Kao, and C. Hu, "Neural network-based and modeling with high accuracy and potential model speed," *IEEE Transactions on Electron Devices*, vol. 69, no. 11, pp. 6476–6479, 2022.

[9] M. Li, O. İrsoy, C. Cardie, and H. G. Xing, "Physics-inspired neural networks for efficient device compact modeling," *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 2, pp. 44–49, 2016.

[10] J. Wang, Y.-H. Kim, J. Ryu, C. Jeong, W. Choi, and D. Kim, "Artificial neural network-based compact modeling methodology for advanced transistors," *IEEE Transactions on Electron Devices*, vol. 68, no. 3, pp. 1318–1325, 2021.

[11] Y. Zhang, G. He, K.-T. Tang, Y. Li, and G. Wang, "Gem: A generalized memristor device modeling framework based on neural network for transient circuit simulation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 3, pp. 834–846, 2023.

[12] G. Gildenblat, *Compact modeling: Principles, techniques and applications*, 01 2010.

[13] V. Realinho, J. Machado, L. Baptista, and M. Martins, "Predicting student dropout and academic success," *Data*, vol. 7, no. 11, p. 146, 2022, academic Editors: Antonio Sarasa Cabezuelo and Ramón González del Campo Rodríguez Barbero. [Online]. Available: https://doi.org/10.3390/data7110146

[14] T. G. Dietterich, "Ensemble methods in machine learning," *International Workshop on Multiple Classifier Systems*, pp. 1–15, 2000.

[15] R. E. Schapire, "A brief introduction to boosting," *IJCAI*, vol. 99, no. 2, pp. 1401–1406, 1999.

Chapter A

Machine Learning Implementation

A.1 Multilayer Perceptron (MLE)

```
class MLE(nn.Module):
    def __init__(self):
        super(MLE, self).__init__()
        self.features = nn.Sequential(
            nn.Linear(3, 30),  # 3 input, first layer 30 neurons
            nn.ReLU(inplace=True),
            nn.Linear(30, 10),  # second layer 10 neurons
            nn.ReLU(inplace=True),
            nn.Linear(10, 1),   # 1 output
        )

    def forward(self, x):
        x = self.features(x)
        x = x.squeeze(-1)
        return x
```

Listing A.1: PyTorch implementation of MLE

A.2 Graph Convolutional Network (FatGCN)

```
class FatGCN(nn.Module):
    def __init__(self, in_channels):
        super(FatGCN, self).__init__()
```

```
        self.conv1 = GCNConv(in_channels, 64)
        self.conv2 = GCNConv(64, 128)
        self.conv3 = GCNConv(128, 256)
        self.norm = nn.LayerNorm(256)
        self.mlp = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x, edge_index, batch):
        x = F.relu(self.conv1(x, edge_index))
        x = F.relu(self.conv2(x, edge_index))
        x = F.relu(self.conv3(x, edge_index))
        x = self.norm(x)
        x = global_mean_pool(x, batch)
        out = self.mlp(x)
        return out
```

Listing A.2: FatGCN implementation

A.3 Random Forest

```
rf_model = RandomForestRegressor(
    n_estimators=50,   # Reduced number of trees
    max_depth=10,      # Limited maximum depth
    random_state=42,
)

param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5]
}
```

Listing A.3: Random Forest implementation with hyperparameter tuning

A.4 XGBoost

```
xgb_model = XGBRegressor(random_state=42)
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
grid_search = GridSearchCV(
    estimator=xgb_model,
    param_grid=param_grid,
    cv=5,
    scoring='neg_mean_absolute_error',
    n_jobs=-1,
    verbose=2
)
grid_search.fit(X_train, y_train)
best_xgb_model = grid_search.best_estimator_
```

Listing A.4: XGBoost implementation with grid search